Match-3 Falling Bricks Game - Interview Preparation Guide

1. OVERALL ARCHITECTURE & APPROACH

High-Level Design

"I approached this as a functional, immutable state-based game engine."

- **Separation of Concerns**: Game logic, state management, rendering, and I/O are completely separated
- **Immutable State**: GameState and Field never mutate - they create new instances
- **Pure Functions**: Most methods have no side effects, making them predictable and testable
- **Domain-Driven Design**: Models real-world concepts (Brick, Field, Position, Block)

Why This Architecture?

- **Testability**: Pure functions are trivial to unit test
- **Thread Safety**: Immutable objects are inherently thread-safe
- **Debugging**: Easy to trace state changes
- **Time Travel**: Could implement undo/redo easily
- **Predictability**: Same input always produces same output

---

2. DESIGN PATTERNS USED

Value Object Pattern (Position, Block)

```java
record Position(int row, int col) {
    Position move(int deltaRow, int deltaCol) {
        return new Position(row + deltaRow, col + deltaCol);
    }
}
```

Why?

- Immutable by default
- Value equality (two positions with same row/col are equal)
- Compact syntax with records

Immutable Object Pattern (Brick, Field, GameState)

```java
public Brick moveLeft() {
    return new Brick(orientation, symbols, position.move(0, -1));
}
```

Why?

- Thread-safe
- No defensive copying needed
- Prevents bugs from unexpected mutations
- Enables functional composition

Strategy Pattern (Command Processing)

```java
private GameState processCommands(GameState state, List<Character> commands) {
    return commands.stream()
        .reduce(state, GameState::processCommand, (s1, s2) -> s2);
}
```

Why?

- Commands are data, not code
- Easy to add new commands
- Can validate, queue, or replay commands

State Pattern (GameState transitions)

```java
public GameState moveDownOrPlace() {
    // Returns new state based on current state
}
```

Why?

- Clear state transitions
- No hidden state changes
- Easy to reason about game flow

Factory Pattern (Brick creation)

```java
private Optional<Brick> parseBrick(String input, int width, int height) {
    // Complex brick construction logic
}
```

Why?

- Encapsulates complex creation logic
- Validates input during construction
- Returns Optional for failure handling

---

3. FUNCTIONAL PROGRAMMING TECHNIQUES

Stream API Usage

```java
// Example 1: Command processing
commands.stream()
    .reduce(state, GameState::processCommand, (s1, s2) -> s2);

// Example 2: Validation
brick.getBlocks().stream()
    .allMatch(block -> isValidPosition(block.pos()) && !isOccupied(block.pos()));

// Example 3: Symbol parsing
parts[1].chars()
    .mapToObj(c -> (char) c)
    .filter(VALID_SYMBOLS::contains)
    .toList();
```

Benefits:

- Declarative vs imperative
- Chainable operations
- Lazy evaluation
- No explicit loops

Optional for Null Safety

```java
public Optional<Brick> getActiveBrick() { return activeBrick; }

activeBrick.ifPresent(brick ->
    brick.getBlocks().forEach(block -> display.put(block.pos(), block.symbol()))
);
```

Why?

- Explicit handling of absence
- Eliminates NullPointerExceptions
- Forces caller to handle both cases

Pattern Matching (Switch Expressions)

```java
Brick moved = switch (command) {
    case 'L' -> current.moveLeft();
    case 'R' -> current.moveRight();
    case 'D' -> dropBrick(current);
    default -> current;
};
```

Benefits:

- Exhaustive checking
- Expression-based (returns value)
- More readable than if-else chains

Method References & Lambda

```java
brick.getBlocks().forEach(block -> newCells.put(block.pos(), block.symbol()));
toRemove.forEach(newCells::remove);
```

---

4. TEST-DRIVEN DEVELOPMENT (TDD) READINESS

How This Code Supports TDD

1. Pure Functions - Easy to Test

```java
// Test example:
@Test
void testBrickMoveLeft() {
    Brick brick = new Brick(HORIZONTAL, List.of('~','^','*'), new Position(0, 5));
    Brick moved = brick.moveLeft();
    assertEquals(new Position(0, 4), moved.getPosition());
}
```

2. No Hidden Dependencies

- No static state
- No global variables
- Dependencies passed explicitly

3. Predictable Behavior

```java
// Same input = Same output, always
Field field = new Field(8, 12);
assertTrue(field.isValidPosition(new Position(0, 0)));
assertFalse(field.isValidPosition(new Position(-1, 0)));
```

4. Isolated Components

- Field doesn't know about Game
- Brick doesn't know about Field
- Each class can be tested independently

Test Categories I Would Write

Unit Tests:

- Position movement calculations
- Brick orientation and movement
- Field validation logic
- Match detection algorithm
- Command processing

Integration Tests:

- Complete game flow
- Brick placement and matching
- State transitions

Property-Based Tests:

- Valid positions always stay in bounds
- Placed bricks never overlap
- Matches always have 3+ consecutive blocks

---

5. KEY ALGORITHMS EXPLAINED

Match Detection Algorithm

```java
public Field removeMatches() {
    Set<Position> toRemove = new HashSet<>();

    // Horizontal matches
    for (int row = 0; row < height; row++) {
        for (int col = 0; col <= width - 3; col++) {
            toRemove.addAll(checkMatch(row, col, 0, 1));
        }
    }

    // Vertical matches
    for (int col = 0; col < width; col++) {
        for (int row = 0; row <= height - 3; row++) {
            toRemove.addAll(checkMatch(row, col, 1, 0));
        }
    }
    // ...
}
```

Complexity Analysis:

- **Time**: O(width × height) - must check every potential match start
- **Space**: O(matches) - stores positions to remove
- **Optimization**: Could use sliding window, but current approach is clear and sufficient

Why This Approach?

- Delta direction (rowDelta, colDelta) makes it generic
- Same logic handles horizontal and vertical
- Collects all matches before removing (prevents partial removal)

Drop Algorithm

```java
private Brick dropBrick(Brick brick) {
    Brick dropped = brick;
    while (field.canPlaceBrick(dropped.moveDown())) {
        dropped = dropped.moveDown();
    }
    return dropped;
}
```

Complexity:

- **Time**: O(height) worst case - brick at top drops to bottom
- **Space**: O(1) - only stores current brick state

Alternative Approaches Considered:

- Binary search for landing position (more complex, minimal benefit)
- Calculate exact landing row (harder with arbitrary placed bricks)

---

6. DATA STRUCTURES CHOICES

HashMap for Field Cells

```java
private final Map<Position, Character> cells;
```

Why HashMap over 2D Array?

- ✅ Sparse data structure (most cells empty)
- ✅ O(1) lookup, insertion, deletion
- ✅ Easy to iterate over only occupied cells
- ✅ No null checks needed
- ❌ Slightly more memory overhead per entry

When Would I Use Array Instead?

- Dense fields (most cells occupied)
- Need to iterate all cells frequently
- Memory is critical constraint

LinkedList for Brick Queue

```java
private final Queue<Brick> brickQueue;
```

Why Queue Interface?

- Clear intent: FIFO ordering
- Easy to swap implementation (LinkedList, ArrayDeque)
- Standard Java Collections API

---

7. IMMUTABILITY DEEP DIVE

Why Immutability Throughout?

Example: Field Operations

```java
public Field placeBrick(Brick brick) {
    Map<Position, Character> newCells = new HashMap<>(cells);
    brick.getBlocks().forEach(block -> newCells.put(block.pos(), block.symbol()));
    return new Field(width, height, newCells);
}
```

Benefits:

1. **Thread Safety**: Multiple threads can read without synchronization
2. **Cache Safety**: Can safely cache/memoize results
3. **Temporal Reasoning**: Can keep history of all states
4. **Debugging**: State at any point is complete snapshot
5. **Testing**: No setup/teardown needed between tests

Trade-offs:

- ❌ More memory allocations
- ❌ More garbage collection
- ✅ But: Modern JVMs optimize this well
- ✅ And: Clarity and correctness trump minor performance costs

---

8. ERROR HANDLING & VALIDATION

Input Validation Strategy

1. Fail Fast with Optional

```java
private Optional<Brick> parseBrick(String input, int width, int height) {
    // Returns empty Optional instead of throwing
}
```

2. Command Filtering

```java
.filter(VALID_COMMANDS::contains)
.limit(2)
```

Why? Silently ignore invalid input (as per requirements)

3. Boundary Checking

```java
public boolean isValidPosition(Position pos) {
    return pos.row() >= 0 && pos.row() < height &&
           pos.col() >= 0 && pos.col() < width;
}
```

What I Would Add in Production

- Custom exceptions (InvalidBrickException, etc.)
- Logging (SLF4J)
- Input sanitization
- More descriptive error messages

---

9. GAME LOOP ARCHITECTURE

Frame-Based Processing

```java
while (!state.isGameOver()) {
    displayGame(state);                    // 1. Render
    List<Character> commands = getCommands(); // 2. Input
    state = processCommands(state, commands); // 3. Process
    state = state.moveDownOrPlace();       // 4. Update

    if (state.needsNewBrick()) {
        state = state.spawnNextBrick();    // 5. Spawn
    }
}
```

This is the Classic Game Loop Pattern:

1. Render current state
2. Get player input
3. Process input
4. Update game state
5. Check win/loss conditions

Separation of Concerns:

- Game class: I/O and orchestration
- GameState class: Pure game logic
- Field class: Spatial logic

---

10. EXTENSIBILITY & FUTURE ENHANCEMENTS

How Easy to Add Features?

1. New Commands (e.g., Rotate)

```java
// Just add to GameState:
case 'T' -> current.rotate();
```

2. Scoring System

```java
// Add to GameState:
private final int score;

public GameState removeMatches() {
    int matchCount = toRemove.size();
    return new GameState(newField, brickQueue, activeBrick,
                         gameOver, score + matchCount * 10);
}
```

3. Power-ups

```java
// Add to Block:
record Block(Position pos, char symbol, Optional<PowerUp> powerUp) {}
```

4. Multiplayer

- Already thread-safe due to immutability
- Each player gets own GameState
- Can send state over network (serializable)

5. AI Player

```java
// Can explore all possible moves:
List<GameState> possibleStates = List.of('L', 'R', 'D')
    .stream()
    .map(cmd -> state.processCommand(cmd))
    .toList();
```

---

11. PERFORMANCE CONSIDERATIONS

Current Complexity

Per Frame:

- Command processing: O(commands) = O(1) since max 2
- Brick movement: O(1)

- Match detection: O(width × height)
- Rendering: O(width × height)

Overall: O(width × height) per frame - acceptable for console game

Optimizations If Needed

1. Incremental Match Detection

- Only check around newly placed blocks
- Reduces from O(w×h) to O(1) in best case

2. Dirty Region Rendering

- Only re-render changed areas
- Keep previous frame buffer

3. Object Pooling

- Reuse Position/Block objects
- Trade immutability for performance (not recommended unless proven bottleneck)

Rule: "Premature optimization is the root of all evil" - Donald Knuth

---

12. CODE QUALITY METRICS

What Makes This Code High Quality?

1. Readability

- Descriptive names (processCommands, canPlaceBrick)
- Small methods (< 20 lines each)
- Single responsibility per method

2. Maintainability

- Low coupling between classes
- High cohesion within classes
- Easy to understand data flow

3. Testability

- Pure functions

- No hidden state
- Deterministic behavior

4. Robustness

- No null pointer exceptions (using Optional)
- Immutable state prevents bugs
- Validation at boundaries

---

13. COMMON INTERVIEW QUESTIONS & ANSWERS

Q: "Why use records instead of classes?"

A: Records are perfect for value objects:

- Immutable by default
- Automatic equals/hashCode based on values
- Concise syntax
- Clear intent: this is data, not behavior
- Position(3, 5) == Position(3, 5) automatically

Q: "What if the field is very large (1000x1000)?"

A: Current HashMap approach scales well:

- Only stores occupied cells (sparse)
- O(1) access time
- Memory proportional to placed blocks, not field size
- For 1000×1000 with 100 blocks: ~100 entries, not 1M

Q: "How would you add undo functionality?"

A: Very easy with immutable state:

```java
Stack<GameState> history = new Stack<>();
history.push(state); // Before each action
state = history.pop(); // To undo
```

Q: "Why not use inheritance?"

A: Composition over inheritance:

- GameState HAS-A Field (not IS-A)
- Brick HAS-A Position (not IS-A)
- More flexible, less coupling
- No fragile base class problem

Q: "How do you handle concurrent players?"

A: Immutability makes it trivial:

- Each player has own GameState instance
- No shared mutable state
- No synchronization needed for reads
- Can use message passing for updates

Q: "What would you test first?"

A: Bottom-up TDD approach:

1. Position movement (simplest)
2. Brick creation and movement
3. Field validation
4. Match detection (most complex algorithm)
5. GameState transitions
6. Integration tests

---

14. TALKING POINTS DURING CODE WALKTHROUGH

Opening Statement

"I designed this as a functional, immutable state-based game engine. The core principle is that game state never mutates - instead, each operation returns a new state. This makes the code highly testable, thread-safe, and easy to reason about."

When Showing GameState Class

"This is the heart of the application. Notice how every method returns a new GameState rather than modifying this one. This immutability means I can safely pass state around, keep history, or even implement time-travel debugging."

When Showing Match Detection

"I chose a straightforward $O(n^2)$ algorithm here because clarity matters more than premature optimization. The delta-based approach makes it easy to handle both horizontal and vertical matches with the same logic."

When Showing Stream API Usage

"I leveraged Java's functional APIs throughout - streams, optionals, method references. This makes the code more declarative: we describe *what* we want, not *how* to get it."

When Discussing Trade-offs

"Every design decision has trade-offs. Immutability costs us some memory allocations, but gains us safety, testability, and maintainability. For a game of this scale, that's absolutely the right trade-off."

---

15. RED FLAGS TO AVOID

Don't Say:

- ❌ "I just wrote whatever worked"
- ❌ "I'm not sure why I did it this way"
- ❌ "Testing isn't important for this"
- ❌ "Performance doesn't matter"

Do Say:

- ✅ "I chose X because it optimizes for Y"
- ✅ "I considered A vs B, and picked A because..."
- ✅ "This design makes testing straightforward"
- ✅ "I balanced performance with readability"

---

16. WRAP-UP STATEMENT

"This solution demonstrates modern Java best practices: immutability for safety, functional programming for clarity, and thoughtful design patterns for maintainability. Every choice was deliberate - from using records for value objects to HashMap for sparse data. The code is production-ready in terms of structure and quality, and could easily scale to more complex requirements like scoring, multiplayer, or AI opponents."

---

17. QUESTIONS TO ASK INTERVIEWER (SHOW ENGAGEMENT)

1. "What's your team's approach to balancing functional vs object-oriented paradigms?"
2. "How do you typically handle state management in your game systems?"
3. "What testing strategies does your team find most effective?"
4. "Are there any specific performance constraints I should consider for this type of application in your environment?"

---

18. PRACTICE EXERCISES BEFORE INTERVIEW

1. **Code Walkthrough**: Explain each class in 2 minutes
2. **Trace Execution**: Walk through one complete frame
3. **Design Defense**: Justify each design pattern choice
4. **Extension Discussion**: Propose 3 new features and how to add them
5. **Debug Scenario**: Explain how you'd debug "matches not clearing"

Good luck with your interview! 🚀